Simple Guide: Distributed Food Ordering

Kafka (KRaft) + Spark Batch · Big Data 2026 (UE24CS343AB2) · My role: **VM1** (Producer 1, Consumer 2, Spark Job 2)

1. The whole assignment in one minute

A food delivery app has a file of **62 orders** (`order.jsonl`). Some of them are junk: broken records, cancelled orders, bad timestamps, duplicates. We have to:

- **Send** the raw orders through Kafka (VM1 -> VM4).
- **Clean** them and add delivery numbers with Spark (VM4). 62 raw -> **44 good orders**.
- **Send** the cleaned orders back through Kafka (VM4 -> VM1).
- **Summarise** them with Spark (VM1) into two reports: one per restaurant, one per city.
- **Prove** that Kafka keeps working when one broker (VM3) is switched off.

Analogy. Kafka is a **post office** that stores letters (messages) in labelled mailboxes (*topics*). It keeps two copies of every letter, one in each branch (VM2 and VM3), so if one branch closes the letters are still there. Spark is the **calculator** each side uses on its own desk after collecting the mail.

The 4 machines

VM	Who	What it does in plain words
VM1 (me)	Producer 1, Consumer 2, Spark Job 2	Posts the raw orders into Kafka. Later collects the cleaned orders and makes the two final CSV reports.
VM2	Kafka broker 1 + controller	A Kafka server, and also the **boss** of the cluster: it knows which servers are alive and who is in charge of each piece of data.
VM3	Kafka broker 2	The second Kafka server. Holds a backup copy of everything. This is the one we switch off in the failure test.
VM4	Consumer 1, Spark Job 1, Producer 2	Collects the raw orders, cleans them and adds metrics with Spark, then posts the cleaned orders back into Kafka.

The flow, step by step

```
STAGE 1                                      STAGE 2
VM1  producer1.py                            VM4  producer2.py
      |  topic: orders  (62 msgs)                   |  topic: order_metrics (44 msgs)
      v                                             v
VM4  consumer1.py -> orders_landed.jsonl     VM1  consumer2.py -> order_metrics_landed.jsonl
      v                                             v
VM4  spark_job1.py -> order_metrics.jsonl    VM1  spark_job2.py -> restaurant_summary.csv
      (clean + enrich, 62 -> 44)                                  -> city_summary.csv

Both topics live on the Kafka cluster = VM2 + VM3 (connected over ZeroTier)
Each topic: 2 partitions, replication factor 2
```

Rules from the spec (the things examiners check)

- Spark never reads Kafka directly. A consumer first saves the data to a local `.jsonl` file; Spark reads that file.
- No HDFS. Everything is on the normal local disk (`outputs/`).
- VM4 must not copy its file to VM1. It must go through Kafka (topic `order_metrics`).
- Both topics: **2 partitions, replication factor 2**, same cluster.

- Failure test: stop **VM3**, never VM2 (VM2 is the only boss).
- Exact duplicates counted once in both Spark jobs. All calculated numbers rounded to 2 decimals. Nothing hardcoded.

2. My part: VM1, explained simply

I run three programs, in this order: **producer1.py** (start), then later **consumer2.py** and **spark_job2.py** (end). Two shared helper files sit behind them: `publish.py` and `landing.py`.

2.1 producer1.py: post the raw orders

Reads `order.jsonl` line by line and sends each line to topic **orders**. All of the real work is in the shared helper `publish.py`.

- **Sends every line as-is, junk included.** The producer only delivers; cleaning is Spark Job 1's job. If both did some cleaning, the rules would be split across two places.
- **No key.** To get a key (order_id) it would have to read the JSON, and some lines are broken. Without a key Kafka spreads messages randomly over the 2 partitions (e.g. 31 / 31, and the split changes each run).
- **acks="all"**: Kafka only says "received" once *every* in-sync copy has the message. This is the safest setting.
- **retries=10**: if something hiccups (e.g. VM3 goes down mid-send), it tries again automatically.
- **linger_ms=10**: waits up to 10 ms to bundle messages together, so fewer network trips.
- It waits for confirmation of every message (`flush()` + `f.get()`), so the printed "62 sent" means 62 *confirmed*, along with which partition each one landed in.

2.2 consumer2.py + landing.py: collect the cleaned orders

Reads *everything* from topic **order_metrics** and saves it to `order_metrics_landed.jsonl` (44 lines). VM4's consumer1.py is the same code with topic **orders**.

- **Reads all partitions itself** (`assign()`, no consumer group). A consumer group remembers where it stopped, so a second run would get nothing. We want the *whole* topic every time.
- **Always starts from the beginning** (`seek_to_beginning`, offset 0).
- **Knows when to stop.** Normal consumers run forever. Ours stops once it has reached the last message of every partition *and* nothing new has arrived for 10 s (`--idle-timeout`).
- **Can start early.** `--wait-timeout` (default 120 s, 600 s in the demo) lets it wait for VM4 to publish.
- **Overwrites the file** (`"w"` mode), so running it twice doesn't double the data.
- Prints how many records came from each partition: **16 from partition 0, 28 from partition 1**.

2.3 spark_job2.py: make the two reports

Reads `order_metrics_landed.jsonl` and writes **restaurant_summary.csv** (6 rows) and **city_summary.csv** (4 rows).

Step	What happens	Why
1. Read	Read the file with a fixed schema (types given up front)	Types are guaranteed and the file is read only once
2. Dedup	`dropDuplicates` on all 15 columns	Spec: exact duplicates count once. Removes 0 here because Job 1 already did it, but it protects against a producer being run twice
3. Group	groupBy restaurant_id, then groupBy city	The two analytics the spec asks for

Step	What happens	Why
4. Compute	count, averages, delayed count (rows where status = DELAYED), delay_rate_pct = 100 x delayed / count	delay_rate is worked out from the exact counts, and rounded only once at the end
5. Round	`F.round(..., 2)` on every calculated number	Spec rule
6. Sort + write	Sort the rows, collect them (just 10 rows) and write with Python's csv module	Spark's own writer makes a *folder* of part-files, but the spec wants one file with an exact name

`.cache()` keeps the deduplicated data in memory because it is used by two reports. Without it Spark would recompute it for each one.

2.4 My outputs (know these by heart)

restaurant	orders	avg amount	avg prep	avg delivery	delayed	delay %
R01	8	548.5	13.13	22.0	4	50.0
R02	8	539.25	13.38	27.0	8	100.0
R03	7	664.0	13.86	32.0	4	57.14
R04	7	639.57	12.14	37.0	7	100.0
R05	7	615.14	12.0	42.0	3	42.86
R06	7	712.14	13.43	47.0	7	100.0

city	orders	avg amount	avg delivery	delayed	delay %
Bengaluru (R01+R02)	16	543.88	24.5	12	75.0
Hyderabad (R06)	7	712.14	47.0	7	100.0
Mumbai (R03+R04)	14	651.79	34.5	11	78.57
Pune (R05)	7	615.14	42.0	3	42.86

Easy cross-check: each restaurant is in exactly one city, so Pune = R05 and Hyderabad = R06. Bengaluru = R01 + R02: 8+8 = 16 orders, 4+8 = 12 delayed, 12/16 = 75%. Order counts add up to 44; delayed orders add up to 33 of 44. Example: R05 delay % = 100 x 3 / 7 = 42.857 -> **42.86**.

2.5 My demo commands

```
nc -zv $VM2 9092 ; nc -zv $VM3 9092                      # can I reach both brokers?
python producer/producer1.py --input order.jsonl         # 62 published
python consumer/consumer2.py --wait-timeout 600 \
    --output outputs/order_metrics_landed.jsonl          # 44 landed
python spark/spark_job2.py --output-dir outputs \
    --input outputs/order_metrics_landed.jsonl           # 6 + 4 rows
python tests/verify_outputs.py --raw order.jsonl \
    --metrics outputs/order_metrics_landed.jsonl         # OK
cat outputs/restaurant_summary.csv outputs/city_summary.csv
# failure test, after VM3 is stopped (talk to VM2 only):
python consumer/consumer2.py --bootstrap-servers $VM2:9092 \
    --output outputs/failover_test/order_metrics_landed.jsonl   # still 44
```

3. My teammates' parts (short version)

VM4: Consumer 1, Spark Job 1, Producer 2 (the cleaning step)

- **consumer1.py**: same as my consumer2, but reads topic **orders** -> `orders_landed.jsonl` (62 lines).
- **spark_job1.py**: the heart of the project. It **throws away bad records**, **removes duplicates**, and **adds 7 new columns**. Output: `order_metrics.jsonl` (44 lines).
- **producer2.py**: posts those 44 lines to topic **order_metrics**, **keyed by order_id**. The same key always goes to the same partition, so the split is always **16 / 28**.

Cleaning funnel: 62 -> 48 -> 44

Thrown out because...	Count
status is not DELIVERED (T04X0001-6: cancelled, pending, failed, refunded)	6
no order_id (2 "invalid_order_record" lines)	2
missing fields ({"order_id":"BROKEN"} and a T040002 copy without a city)	2
bad timestamp (T040005 "bad", T040003 "not-a-timestamp")	2
distance_km <= 0 (T040001: -2) and order_amount <= 0 (T040004: 0)	2
Total dropped -> 48 valid	**14**
Exact duplicates removed (T040006, T040017, T040028, T040039) -> **44**	4

New columns Spark Job 1 adds (all in minutes, rounded to 2 decimals):

Column	Formula
preparation_time_min	picked_up - accepted
delivery_time_min	delivered - picked_up
fulfillment_time_min	delivered - order_timestamp
delivery_speed_kmph	distance_km / (delivery_time_min / 60)
delay_min	max(0, delivered - expected)
delivery_status	ON_TIME if delay = 0, else DELAYED
delay_category	0 ON_TIME | up to 15 SLIGHT | up to 30 MODERATE | over 30 SEVERE

Midnight trick: it uses full date+time, not just the time of day. T040023 picked up 23:20 and delivered 00:02 the next day = **42 min**, not -1398. The time zone is set to UTC so daylight saving can't shift anything.

VM2: Broker 1 + KRaft controller (the boss)

- Runs Kafka with `process.roles=broker,controller`, `node.id=1`.
- **KRaft** = Kafka manages itself, with no separate ZooKeeper. The controller keeps the cluster's "register" (which brokers are alive, which topics exist, who leads each partition).
- Port **9092** = normal Kafka traffic, port **9093** = controller traffic (only VM2 has it).
- Generates the **cluster ID** once; VM3 must use the same ID to join.
- `min.insync.replicas=1`: writes still succeed with only one copy alive (needed while VM3 is off).
- `auto.create.topics.enable=false`: topics are only created on purpose, so a typo can't create a wrong topic.

VM3: Broker 2 (the backup)

- Runs Kafka with `process.roles=broker` only, `node.id=2`.

- Points at VM2:9093 to join and is formatted with **VM2's cluster ID** (it must not create its own cluster).
- Holds the second copy of every partition. In the failure test, **this is the machine that gets stopped**.

The failure test (whole team)

```
Before (both up):    orders P0  Leader: 2  Replicas: 2,1  Isr: 2,1
                     orders P1  Leader: 1  Replicas: 1,2  Isr: 1,2
After stopping VM3:  orders P0  Leader: 1  Replicas: 2,1  Isr: 1     <- leader moved to 1
                     orders P1  Leader: 1  Replicas: 1,2  Isr: 1
Restart VM3 -> Isr back to 2,1 / 1,2.
```

What the TA checks, in order:

- **A. Before:** describe shows 2 partitions, RF 2, Isr has both brokers. Under-replicated list is empty.
- **B. Stop VM3:** Leader 1 and Isr 1 everywhere. Only broker id 1 is listed.
- **C. Pipeline continues with VM3 down:** rerun the whole pipeline through VM2 only, into `outputs/failover_test/`. Producing still works (acks=all + min.insync.replicas=1).
- **D. Restart VM3:** Isr back to both brokers, under-replicated list empty again. Rerun the pipeline into `outputs/recovery_test/`.

Why the counts grow: Kafka keeps old messages, so each run adds to the topics: orders 62 -> 124 -> 186, order_metrics 44 -> 88 -> 132. Those extra copies are exact duplicates, so Job 1 still outputs **44** and the CSVs are byte-identical (checked with `diff`). Full commands: demo_commands.md, step 10.

Leader = the broker currently serving that partition. **Replicas** = where copies are *supposed* to live (doesn't change). **Isr** = copies that are currently up to date (shrinks to just 1 while VM3 is off).

4. Key words in one line each

Term	Simple meaning
Broker	A Kafka server that stores messages (VM2, VM3).
Controller	The boss broker that tracks who is alive and picks leaders (only VM2).
KRaft	Kafka's built-in way of electing and storing the boss's records. It replaced ZooKeeper.
Topic	A named mailbox: *orders*, *order_metrics*.
Partition	A topic is split into pieces (we use 2) so work can be shared. Order is kept only inside one partition.
Offset	A message's position number inside a partition (0, 1, 2 ...).
Replication factor 2	Every partition is stored twice, once on each broker.
Leader / follower	The leader copy handles reads and writes; the follower copies it.
ISR	In-Sync Replicas: the copies that are fully up to date.
acks=all	The producer gets "OK" only when all in-sync copies have the message.
Bootstrap servers	The first broker address(es) a client contacts to discover the whole cluster.
advertised.listeners	The address a broker tells clients to use. It must be the ZeroTier IP, not localhost.
ZeroTier	A virtual LAN, so the 4 laptops on different home networks can reach each other.
local[*]	Spark runs on this one machine, using all CPU cores.
Lazy evaluation	Spark only plans the work; it runs when an action (count, collect) is called.

5. Top viva questions, short answers

Q. Why use Kafka instead of just copying files?

It separates the stages (VM1 and VM4 never talk directly), stores the data reliably in two copies, and lets any stage re-read the data. The spec also forbids copying the file.

Q. Why is Producer 1 unkeyed but Producer 2 keyed?

Raw lines may be broken JSON, so Producer 1 doesn't read them to find a key. Cleaned lines are always valid, so Producer 2 keys by order_id, which gives a fixed partition split (16/28).

Q. Why doesn't your consumer use a consumer group?

Groups remember where they stopped, so a second run would read nothing. Ours is a batch job that must copy the whole topic every time, so it reads all partitions from offset 0 and commits nothing.

Q. How does your consumer know when to stop?

When its position equals the last offset of every partition and nothing new has arrived for 10 s.

Q. What happens when VM3 stops?

VM3 stops sending heartbeats, the controller on VM2 removes it from the ISR and makes broker 1 the leader everywhere. Broker 1 already has a full copy (RF 2), so no data is lost.

Q. Why stop VM3 and not VM2?

VM2 is the only controller. Without it nobody can elect leaders. Real high availability needs 3 controllers.

Q. Can you still produce while VM3 is down?

Yes. acks=all only waits for the in-sync copies (now just broker 1), and min.insync.replicas=1 allows that.

Q. What if Producer 1 runs twice?

The topic has 124 messages, but Spark Job 1 still outputs 44 because duplicates are exact and get removed. Delivery is at-least-once, which is why both jobs dedupe.

Q. Why collect() and write the CSV with Python?

Spark's writer makes a folder of part-files. The spec wants single files with exact names, and the results are tiny (6 and 4 rows).

Q. Why is dropDuplicates in Job 2 if it removes 0?

The spec says both jobs must dedupe. It also protects against Kafka redelivery or a producer re-run.

Q. Why not deduplicate by order_id?

The spec says *exact* duplicates. Choosing between two different records with the same ID would be inventing a business rule.

Q. How do you know the outputs are correct?

tests/verify_outputs.py redoes everything in plain Python (no Spark) and compares every value. It prints "OK: 44 enriched records and both summaries match".

Q. Why Spark for only 62 records?

The assignment is about the architecture. The same code would work on millions of rows or on a real cluster.

6. Numbers to remember

Item	Value
Raw lines	62
Dropped by cleaning	14 -> 48 valid
Exact duplicates removed	4 -> 44 enriched

Item	Value
orders partition split	random, e.g. 31 / 31
order_metrics partition split	always 16 / 28
Restaurants / cities	6 (R01-R06) / 4 (Bengaluru, Hyderabad, Mumbai, Pune)
Delayed orders	33 of 44 (11 on time)
Delay categories	ON_TIME 11, SLIGHT 11, MODERATE 15, SEVERE 7
Highest / lowest delay %	100% (R02, R04, R06, Hyderabad) / 42.86% (R05, Pune)
Fastest / slowest delivery	R01 22.0 min / R06 47.0 min
Node IDs / ports	VM2 = node 1 (9092 + 9093); VM3 = node 2 (9092)
Versions	Kafka 4.3.1 (KRaft), PySpark 4.2.0, kafka-python 2.2.15, Java 17+